

Assignment 2: Multi-Stage Recommendation — Rating Prediction

Table of Contents

- [Task Description](#)
 - [Core Concepts](#)
 - [Dataset](#)
 - [Files Provided](#)
 - [Submission Format](#)
 - [Allowed Tools](#)
 - [Report Requirements](#)
 - [Grading](#)
 - [Tips](#)
 - [Academic Integrity](#)
-

Task Description

You are given user–movie rating data. Each rating is an integer from 1 to 5 with a timestamp.

For each `(user_id, item_id)` pair in `test_pairs.csv`, your task is to **predict the rating the user gave that movie**. The true test ratings are hidden. Your submission is scored by **Root Mean Squared Error (RMSE)** against the hidden ground truth.

You are encouraged to treat this as a **multi-stage recommendation pipeline**:

1. **Retrieval**: a fast, coarse method that identifies candidate items and gives rough scores (e.g., popularity, item-CF with cosine similarity, matrix factorization dot product).
2. **Ranking**: a more precise method that refines the scores for the candidates you actually need to predict (e.g., MF with biases, or a hybrid that mixes collaborative signal with movie metadata).

You don't have to literally split your code into two stages — a single MF model that produces ratings directly is fine — but your report must **discuss** how you would factor the problem into retrieval and ranking, and why.

Core Concepts

1. **Collaborative filtering (CF).** Methods that learn from the rating matrix alone — e.g., neighborhood methods (item-based / user-based similarity) or latent-factor methods (matrix factorization, SVD). These are the natural starting point and come up in lecture.
2. **Content features (encouraged, not required).** `movies.csv` and `users.csv` give you movie metadata and user demographics. A **hybrid** approach that combines collaborative signal with content features usually helps, especially on cold-start items.
3. **Evaluation.** Because your predictions are point estimates of ratings, RMSE is the right metric. Predictions are clipped to `[1.0, 5.0]` before scoring.

Dataset

Movie ratings collected between 2000 and 2003. Ratings are integers on a 1–5 scale, each with a Unix timestamp.

Split Policy

Per-user temporal **leave-last-20%**. For each user, ratings are sorted by `timestamp` ascending and the most recent `ceil(0.20 × n_ratings)` ratings are held out as test. Every user has at least 1 training and 1 test rating. Random seed `172` breaks timestamp ties.

Statistics

	Count
Users (all appear in both train and test)	6,040
Distinct movies in ratings	3,706
Movies in <code>movies.csv</code>	3,883
Training ratings	797,758
Test pairs	202,451
Cold-start items in test (never in train)	40
Cold-start users in test	0

- **Global mean rating (train):** 3.617
- **Train timestamp range:** 2000-04 → 2003-02
- **Test timestamp range:** 2000-04 → 2003-02 (test skews later per-user by construction)

Rating Distribution

Rating	Train count	Train %
1	42,207	5.3%
2	81,748	10.2%
3	204,611	25.6%
4	280,133	35.1%
5	189,059	23.7%

The test distribution skews slightly lower (users' later ratings tend to be more critical). Full histograms are in `split_stats.json`.

Data Schema

`train.csv` — all ratings you may train on.

Column	Type	Description
<code>user_id</code>	int	User identifier (1–6040)
<code>item_id</code>	int	Movie identifier
<code>rating</code>	int	Rating in {1, 2, 3, 4, 5}
<code>timestamp</code>	int	Unix timestamp (seconds)

`test_pairs.csv` — pairs you must predict (no ratings).

Column	Type	Description
<code>user_id</code>	int	User identifier
<code>item_id</code>	int	Movie identifier

`movies.csv` — movie metadata.

Column	Type	Description
item_id	int	Movie identifier
title	string	Movie title (year stripped)
year	int	Release year (may be missing)
genres	string	Pipe-separated genre tags, e.g. Action Comedy

The 18 genres are: Action, Adventure, Animation, Children's, Comedy, Crime, Documentary, Drama, Fantasy, Film-Noir, Horror, Musical, Mystery, Romance, Sci-Fi, Thriller, War, Western.

users.csv — user demographics.

Column	Type	Description
user_id	int	User identifier
gender	string	M or F
age_group	string	under_18 , 18–24 , 25–34 , 35–44 , 45–49 , 50–55 , 56+
occupation	string	One of 21 labels (e.g. programmer , k12_student , retired)

ZIP code from the original dataset is dropped.

Files Provided

File	Purpose
README.md	This file — the full assignment description
train.csv	Training ratings (797,758 rows)
test_pairs.csv	(user, item) pairs you must predict (202,451 rows)
movies.csv	Movie metadata
users.csv	User demographics
sample_submission.csv	A valid submission that predicts the global mean for every pair
split_stats.json	Summary statistics of the train/test split
evaluation_code.py	The exact RMSE scorer the TAs will run
baseline_global_mean.py	Minimum viable submission script

Submission Format

Submit a CSV file named `submission.csv` .

```

user_id,item_id,predicted_rating
1,1,4.28
1,48,3.91
1,588,4.50
...
```

Requirements

1. **Exactly 202,451 rows** — one row for every pair in `test_pairs.csv` , no extras.
2. **Column names must be exactly** `user_id` , `item_id` , `predicted_rating` .
3. Every `(user_id, item_id)` from `test_pairs.csv` must appear exactly once.
4. `predicted_rating` is a **real number** — decimals like `4.28` are not only allowed, they're expected. The scorer does *not* round your prediction to an integer before computing RMSE, so a calibrated real-valued prediction (e.g., `3.9`) typically beats a hard-rounded one (e.g., `4`) on RMSE. Values outside `[1.0, 5.0]` are clipped to the interval by the scorer.
5. Row order doesn't matter.
6. Include the header row.

Note: `sample_submission.csv` only contains the rows for `user_id ∈ {1, 2}` as a tiny preview of the format. Your actual submission must cover **all 202,451 pairs** in `test_pairs.csv`.

How scoring works

The TAs run `evaluation_code.rmse(ground_truth, your_submission)`. This:

1. Merges your predictions with the hidden ground truth on `(user_id, item_id)`.
2. Fails if any test pair has no prediction.
3. Clips predicted ratings to `[1.0, 5.0]`.
4. Returns `sqrt(mean((pred - true)2))`.

You can run the same scorer locally on your own validation split:

```
from evaluation_code import rmse
import pandas as pd
print(rmse(my_val_truth, my_val_predictions))
```

Common mistakes to avoid

- Renaming columns (e.g., `rating` instead of `predicted_rating`).
- Adding `timestamp` or `rank` columns.
- Submitting predictions for pairs **not** in `test_pairs.csv`.
- Missing predictions for some test pairs.
- Integer-only predictions — real-valued is allowed and usually better.

See `sample_submission.csv` for a working example (it predicts the global mean for every pair).

Allowed Tools

You **may** use:

- `numpy`, `pandas`, `scipy` (including `scipy.sparse` and `scipy.sparse.linalg.svds`)
- `sklearn` **utilities only** — e.g., `TfidfVectorizer`, `cosine_similarity`, `train_test_split`, `StandardScaler`, `TruncatedSVD`. These are linear-algebra / feature utilities, not recommenders.
- `matplotlib` / `seaborn` for plots

You **may not** use recsys libraries that implement the algorithms for you:

- `surprise`
- `implicit`
- `lightfm`
- `lenskit`
- Any deep-learning framework used solely to call a pre-built recommender.

Matrix Factorization must be implemented by you (e.g., numpy + SGD, ~40 lines).

Report Requirements

Submit a PDF or Jupyter notebook (`report.pdf` or `report.ipynb`) containing:

1. **Data exploration.** Rating distribution, user-activity distribution, item-popularity distribution, timestamp range of train vs. test. At least 2–3 plots.
2. **Methods.** Implement **at least two** methods with increasing quality, for example:
 - a baseline (global mean, user mean, or item mean),
 - **at least one** of: item-based CF, matrix factorization (Funk SVD), or truncated SVD on the imputed matrix.

For each method, describe the formula, your hyperparameters, and how you tuned them on your own validation split.
3. **Multi-stage discussion.** One section explaining how you would (or did) organize the system as retrieval + ranking. Address: what each stage produces, why the two stages have different cost/quality trade-offs, and which of your methods would play each role.
4. **Hybrid attempt (encouraged).** Incorporate at least one content feature from `movies.csv` (genres, year) or `users.csv` (demographics) and report whether RMSE improved.
5. **Validation RMSE for every method.** Build your own validation split from `train.csv` (temporal, to mirror the test setup) and report RMSE for every method you tried.
6. **Analysis.** At minimum: how do the methods differ on cold-start items? Where does most of the error come from — users with few ratings, items with few ratings, extreme ratings? What would you try next if you had more time?

Grading

Component	Weight
Test RMSE (leaderboard-style, held-out scorer)	25%
Method correctness (code review)	25%
Report quality (exploration, analysis, multi-stage discussion)	40%
Hybrid / bonus experiments	10%

Tips

- **Start with the global-mean baseline.** Run `baseline_global_mean.py`, then score it against your own validation split with `evaluation_code.py`. This confirms your data-loading / submission / scoring pipeline works before you implement anything interesting.
- **Build your own validation split.** Mirror the test setup — do a per-user temporal leave-last-N% on `train.csv`. When done right, your validation RMSE and the TAs' test RMSE should track each other.
- **Tune hyperparameters on your validation split**, not by peeking at the test file.
- **Sanity-check your submission** before uploading: row count = 202,451, columns are exactly `user_id, item_id, predicted_rating`, no NaNs.

Academic Integrity

You must implement your algorithms from scratch. Copying MF / SVD code from public repositories (including Surprise and textbook implementations) will be treated as a violation. Discussing high-level ideas with classmates is fine; sharing code is not.